

GRAPH ANALYTICS – PART 1

Introduction

- What is and why using a Graph
- Graph data science:
 - definition of Graph Analytics, use cases, graph market
 - graphs and machine learning
- Networks and Graph to model real world applications

Graph Theory

- The Bridges of Königsberg
- Graph as nodes and edges:
 - Direct graph, undirect graphs, cycles, multiple edge, simple graph, multigraph, complete graph, degree of a vertex, avg degree of a graph, weighted graphs, bipartite and multipartite monpartite graph, path
 - Vertices: labelled, unlabelled
 - Edges: incoming, outcoming
 - Order and Size of a graph
- Path: directed, undirected path, simple path, cycles,
- Connected graph, disconnected graph, Connected components, strongly connected components
- Weighted graphs
- Monopartite and multipartite graph

Graph storage

- What is a graph: computer science / mathematical definition
- Storing a graph: Edge List, Adjacency Matrix, Adjacency List - Cost of operations
- Graph density: sparse and dense graph – Efficiency in graph storage

Graph Navigation

- Traversing the graph:
 - Walk, Open and Closed walk, Trail, Circuit, Path, Cycle
 - Eulerian and semi-Eulerian graphs
 - Traversable graphs, Hamiltonian Path (traceable graph)
- Operation on a graph
 - Neighbors, InDegree, OutDegree, Reachable nodes
 - Graph Traversal: Depth First Search, Breadth First Search, Shortest Path (weighted/unweighted graph)
 - Shortest Paths: Single source, Single Pair of nodes, all pairs shortest path
 - BFD and Shortest path
 - Algorithm and application for shortest path (Dijkstra, Bellman–Ford, A* search, Floyd–Warshall ...)
- Graph Statistics
 - Connectivity: connected or disconnected graphs
 - Max-Flow Min-Cut Theorem
 - Eccentricity of a node, Radius and Diameter of a graph
 - Graph partition: connected components, strongly connected components, strongly connected directed graphs
 - Conductance
- Graph Structure:
 - Cliques, completed subgraphs
 - K-Core decomposition
 - Graph Structure Analysis: Average Diameter
 - Clustering coefficient, local density, Avg clustering coefficient
 - Closeness of a graph, Wiener index
- Centrality measures:

- Degree centrality, Closeness centrality, Betweenness centrality
- Community Detection (Girvan-Newman Algorithm), Non overlapping vs. overlapping communities
- Importance in Directed Graphs: Proximity Prestige, In Degree prestige, Influence Domain
- Measure Impact: H-Index

Graph Importance

- Basic Concepts:
 - Random walk
 - Transition probability
 - Teleport probability
 - The random surfer
 - Markov Model: adjacency matrix, transition probability matrix
 - Algebraic representation of the Markov model
- Applications / graph structure mining:
 - Citation Network
 - Co-Citation: Co-Citation Matrix and its computation
 - Bibliographic Coupling and its computation
 - Use cases
 - HITS (Hypertext Induced Topic Search):
 - definition of hub and authority
 - Hub/Authority Score
 - Mutual reinforcement relations
 - Hub/Authority Score computation
 - Page Rank
 - How the google's page rank works
 - Importance flow, Matrix formulation, Power Iteration

- Stationary Distribution: Convergence Issues
- Dead-end and spider-traps
- Conditions for the Existence and Uniqueness of Stationary Distribution in Markov Processes
- Personalized Page Rank: Topic-Specific PageRank
 - Bias the random walk
 - Personalized Page Rank as a Proximity Measure
 - ParticleFiltering Approach

Graph databases

- Advantage and potential issues of graph db
- Comparison with other types of databases: key-value, wide-column, document, relational db
- Graph db vs Relational db:
 - Recursive queries (e.g. hierarchies)
 - Union of different result types
 - Paths – modelling the status change (e.g. for example, a river crossing puzzle)
- Type of Graph DB:
 - Property Graph DB
 - Resource Description Framework (RDF) Graph DB
 - Hypergraph DB
 - Spatial Graph DB
 - Temporal Graph DB
 - Property Graph/ Triplestore Hybrid
- Tools:
 - For modelling and storing: Neo4j, Apache TinkerPop
 - For querying: GraphQL, Cypher, Gremlin, GQL standard

Neo4J

- Components of the Neo4j Graph Data Platform:
 - Graph Data Science (GDS)
 - Neo4j AuraDB e AuraDS (Cloud service)
 - Cypher Query Language, GraphQL Library
 - Drivers
 - Neo4J Bloom for visualization
 - Neo4J tools: Browser, Data importer, Desktop, Ops Manager
- Terminology & Syntax
 - Vertex, Properties, Label, Relationships
 - MATCH clause
 - RETURN clause
 - Pattern Syntax, Variable length, path patterns, graph patterns
 - Optional Match, Where
 - Matching Paths: Shortest paths
 - Aggregation and WITH clause
 - Creation CREATE and MERGE
 - Constraints
 - Index
- Graph Landscape
 - graph databases (real time OLTP) and graph compute engines (interactive OLAP)
 - graph database management system: storage (native graph storage) and processing engine (index-free adjacency)
 - Why choosing Graph DB: Performance, Flexibility, Agility and Evolving Data Models

Graph modelling

- Defining project goals
- Building a conceptual data model for the entities and their relationships
- Translating a conceptual data model into a graph data model:
 - Translate entities to vertices
 - Translate relations to edges
 - Identifying relevant relationships
 - Naming edge labels
 - Giving an edge direction
 - Determine edge uniqueness – improper uniqueness
 - Assign properties
 - Check the model
- Comparing graph data modeling concepts to relational data modelling concepts
- Constructing the graph data model